

Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/05/16

- 自编码器(Autoencoders)

自编码器是一种训练来学习恒等函数的神经网络；

自编码器是无监督学习模型；

目标是学习数据的一些有用属性，可用于降维。

- 迁移学习

将为特定任务获得的知识应用于相关任务；

通过提供大数据集学习特征，提高图像分类准确性；

医学成像等领域有限数据可从ImageNet预训练模型获益。

- 自监督学习

使用数据本身生成的标签进行学习；

预训练任务使用伪标签，无需人工标注；

完成预训练后，模型可迁移至下游任务。

- 引言

自编码器 (Autoencoder) 是一种无监督学习模型，用于学习数据的有效表示 (编码)。其核心思想是通过编码器 (Encoder) 将输入数据压缩为低维表示，再通过解码器 (Decoder) 重构原始数据，从而学习数据的本质特征。

- 自编码器的基本结构

- 编码器 (Encoder)

目标：将输入数据 x 映射为低维表示 z (称为编码或潜在表示)。

数学表示：

$$z = f_{\text{enc}}(x) = \sigma(W_e x + b_e)$$

其中：

W_e 为编码器权重矩阵， b_e 为编码器偏置项， σ 为激活函数 (如 ReLU)

- 解码器 (Decoder)

目标：将低维表示 z 重构为原始数据 x 。

数学表示：

$$\hat{x} = f_{\text{dec}}(z) = \sigma(W_d z + b_d)$$

其中：

W_d 为解码器权重矩阵， b_d 为解码器偏置项， σ 为激活函数

- 整体结构

输入层 -> 编码器 -> 潜在表示 z -> 解码器 -> 输出层

自编码器(Autoencoders)

- 自编码器的训练

- 目标函数

自编码器的目标是**最小化输入数据 x 与重构数据 $\hat{x}$ 之间的差异**，通常使用均方误差 (MSE) 作为损失函数：

$$L(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

- 优化过程

通过梯度下降法更新编码器和解码器的参数：

- 前向传播：

计算编码： $z = f_{enc}(x)$

计算重构： $\hat{x} = f_{dec}(z)$

- 计算损失：

$$L(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

- 反向传播：

计算损失对参数的梯度： $\frac{\partial L}{\partial W_e}, \frac{\partial L}{\partial b_e}, \frac{\partial L}{\partial W_d}, \frac{\partial L}{\partial b_d}$

- 参数更新：

$$W_e = W_e - \eta \frac{\partial L}{\partial W_e} \quad b_e = b_e - \eta \frac{\partial L}{\partial b_e} \quad W_d = W_d - \eta \frac{\partial L}{\partial W_d} \quad b_d = b_d - \eta \frac{\partial L}{\partial b_d}$$

- 自编码器的变体

- 稀疏自编码器 (Sparse Autoencoder)

在损失函数中加入稀疏性约束，使得潜在表示 z 具有稀疏性。

目标函数：

$$L(x, \hat{x}) + \lambda \sum_j |z_j|$$

- 去噪自编码器 (Denoising Autoencoder)

在输入数据中加入噪声，训练模型从噪声数据中恢复原始数据。

目标函数：

$$L(x, f_{\text{dec}}(f_{\text{enc}}(\tilde{x})))$$

其中， x 为加入噪声的输入数据。

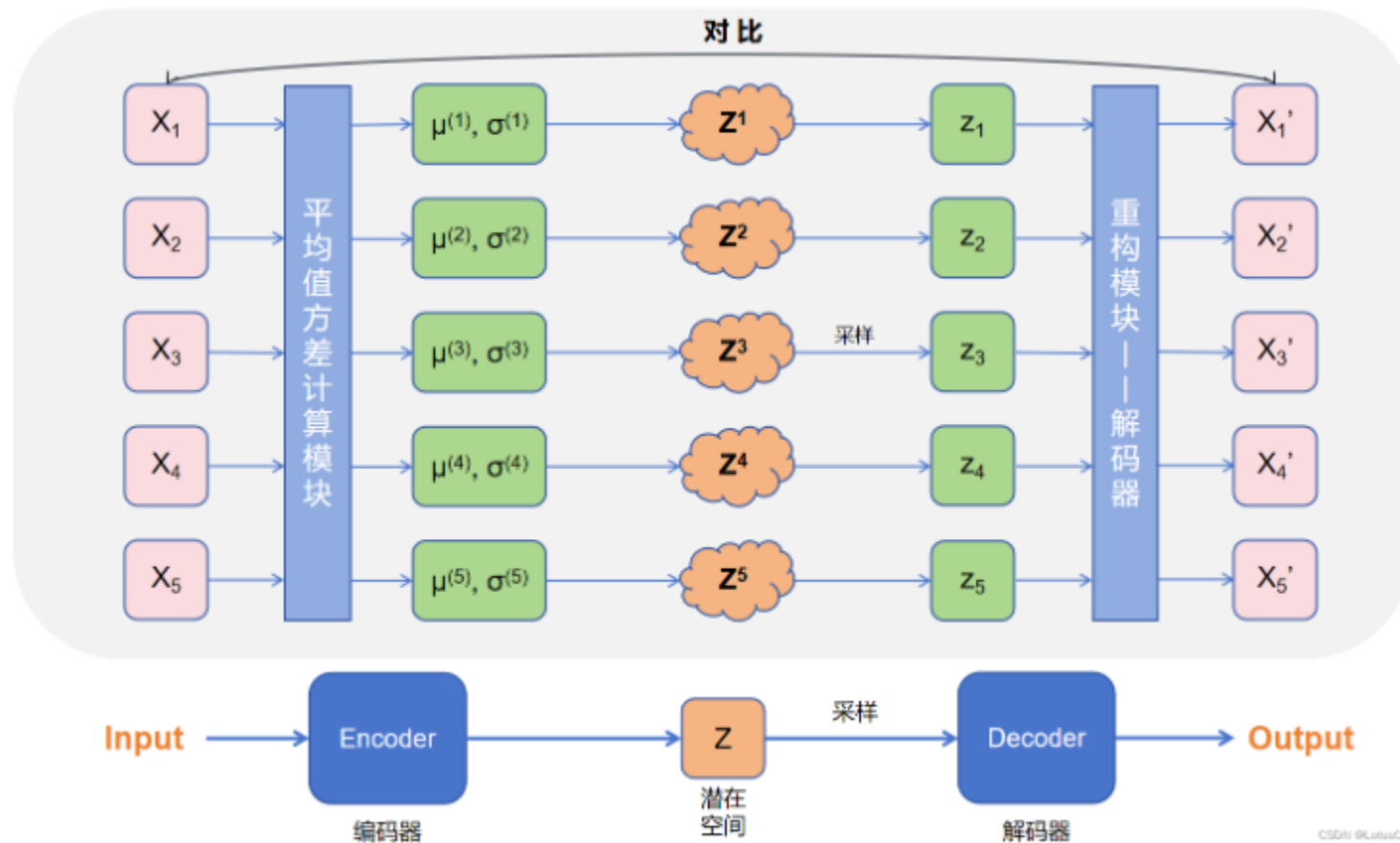
- 变分自编码器 (Variational Autoencoder, VAE)

将潜在表示 z 建模为概率分布，通常使用高斯分布。

目标函数：

$$L(x, \hat{x}) + \text{KL}(q(z|x) || p(z))$$

其中， $q(z|x)$ 为编码器输出的分布， $p(z)$ 为先验分布（通常为标准正态分布）。



CSDN @KulusCL

https://blog.csdn.net/m0_56942491/article/details/136265500?ops_request_misc=%257B%2522request%255Fid%2522%253A%25229767f5ef139aae14eaff212d770cbb36%2522%252C%2522scm%2522%253A%252220140713.130102334..%2522%257D&request_id=9767f5ef139aae14eaff212d770cbb36&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~top_positive~default-1-136265500-null-null.142^v102^pc_search_result_base4&utm_term=VAE&spm=1018.2226.3001.4187

实验1：自编码器(Autoencoders)--理解自编码器实验过程

自编码器实验过程可以理解为四个关键阶段：架构设计、训练过程、评估分析和应用探索。

- 架构设计阶段

在这个阶段，我们根据不同的自编码器类型设计网络结构：

1. 基本自编码器：最简单的瓶颈结构，通过将输入经过编码器压缩到低维潜在空间，再通过解码器重建原始输入。它直接学习数据的压缩表示，没有显式正则化。
2. 欠完备自编码器：有意设计较小的编码维度（本实验中是32维），通过信息瓶颈强制模型学习最重要的特征。它利用维度限制作为隐式正则化手段。
3. 正则化自编码器：在损失函数中添加L1正则化项，鼓励编码向量的稀疏性。稀疏编码通常更具解释性，且可能具有更好的泛化能力。
4. 去噪自编码器：在训练时为输入添加随机噪声，但目标仍是重建原始无噪声图像。这迫使模型学习提取稳健特征的能力，提高抗干扰性。
5. 变分自编码器：将编码器输出解释为概率分布的参数（均值和方差），通过重参数化技巧实现可导的采样过程。它学习的是数据的生成模型，而非简单的编码-解码映射。

实验1：自编码器(Autoencoders)--理解自编码器实验过程

- 训练过程阶段

训练过程中，各类自编码器有不同的学习行为：

1. 基本自编码器：训练曲线通常平稳下降，在MNIST上迅速收敛到低重建误差。CIFAR-10上收敛较慢，最终误差较大。
2. 欠完备自编码器：由于编码维度限制，训练初期重建误差下降较慢，最终收敛值略高于基本自编码器。
3. 正则化自编码器：训练中可观察到编码向量逐渐变得稀疏，某些维度的活跃度明显高于其他维度。总体训练损失比基本自编码器高，因为包含了正则化项。
4. 去噪自编码器：训练过程中，模型先学会去除简单噪声，再逐渐恢复更细微的细节。训练损失下降通常不如基本自编码器快，因为任务本身更具挑战性。
5. 变分自编码器：训练曲线常显示重建损失和KL散度的此消彼长，反映了重建质量与潜在空间正则化之间的权衡。训练初期KL散度通常较低，随着训练进行而增加。

实验1: 自编码器(Autoencoders)--理解自编码器实验过程

- 评估分析阶段

评估自编码器性能需要从多个角度考量:

1. 重建质量: 通过视觉比较原始图像和重建图像, 以及量化的重建误差 (MSE) 评估。
在MNIST上, 大多数模型都能实现高质量重建; 在CIFAR-10上, 质量差异更明显。
2. 潜在空间结构: 通过t-SNE可视化低维表示, 观察不同类别样本的聚类情况。良好的自编码器应在潜在空间中形成清晰的语义结构。
3. 生成能力 (主要针对VAE): 评估从随机噪声生成的样本质量和多样性, 以及潜在空间插值的平滑性。好的VAE应能生成逼真、多样的样本, 且插值应产生自然过渡。
4. 特征质量: 可通过在学习到的特征上训练简单分类器, 评估特征的判别性。有效的自编码器学习到的特征应支持高精度分类。

实验1：自编码器(Autoencoders)--理解自编码器实验过程

- 应用探索阶段

基于实验结果，我们可以确定不同自编码器的最佳应用场景：

1. 基本自编码器：适用于简单的降维和特征提取任务，特别是数据结构相对简单时。
2. 欠完备自编码器：适合需要紧凑特征表示的应用，如数据压缩、高效存储和传输。
3. 正则化自编码器：适用于需要稀疏、可解释特征的场景，如医学图像分析或模式识别。
4. 去噪自编码器：最适合图像去噪、恢复和预处理任务，以及需要对输入噪声鲁棒的特征提取。
5. 变分自编码器：最适合生成任务，如图像合成、数据增强和内容创建，也适用于异常检测和表示学习。

实验1: 自编码器(Autoencoders)--一些思考题

- 自编码器的本质目的是什么? 它与其他无监督学习方法相比有什么优势?
- 为什么欠完备自编码器能学到更有用的特征, 而完备或过完备自编码器容易学到平凡解?
- L1正则化和L2正则化对自编码器的影响有什么不同? 为什么L1正则化更倾向于产生稀疏表示?
- 去噪自编码器为什么能提高模型的鲁棒性? 这种鲁棒性对下游任务有什么意义?
- VAE与传统自编码器的根本区别是什么? 为什么VAE在生成任务上具有优势?
- 潜在空间维度大小对自编码器性能的影响是什么? 在实际应用中应如何选择合适的编码维度?
- 为什么在CIFAR-10数据集上, 大多数自编码器(特别是VAE)的重建质量不如在MNIST上? 应如何改进?



https://blog.csdn.net/Avery123123/article/details/103176283?ops_request_misc=%257B%2522request%255Fid%2522%253A%252274a5be549ab4a38d24fb555c7e4fcf00%2522%252C%2522scm%2522%253A%252220140713.130102334.pc%255Fall.%2522%257D&request_id=74a5be549ab4a38d24fb555c7e4fcf00&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~first_rank_ecpm_v1~rank_v31_ecpm-4-103176283-null-null.142^v102^pc_search_result_base4&utm_term=%E8%87%AA%E5%8A%A8%E7%BC%96%E7%A0%81%E5%99%A8%E8%AF%A6%E8%A7%A3&spm=1018.2226.3001.4187

- 什么是迁移学习

迁移学习 (Transfer Learning) 是一种机器学习方法，就是把为任务 A 开发的模型作为初始点，重新使用在为任务 B 开发模型的过程中。迁移学习是通过从已学习的相关任务中转移知识来改进学习的新任务，虽然大多数机器学习算法都是为了解决单个任务而设计的，但是促进迁移学习的算法的开发是机器学习社区持续关注的话题。迁移学习对人类来说很常见，例如，我们可能会发现学习识别苹果可能有助于识别梨，或者学习弹奏电子琴可能有助于学习钢琴。

找到目标问题的相似性，迁移学习任务就是从相似性出发，将旧领域(domain)学习过的模型应用在新领域上。

- 为什么需要迁移学习？

1. 大数据与少标注的矛盾：虽然有大量的数据，但往往都是没有标注的，无法训练机器学习模型。人工进行数据标注太耗时。
2. 大数据与弱计算的矛盾：普通人无法拥有庞大的数据量与计算资源。因此需要借助于模型的迁移。
3. 普适化模型与个性化需求的矛盾：即使是在同一个任务上，一个模型也往往难以满足每个人的个性化需求，比如特定的隐私设置。这就需要在不同人之间做模型的适配。
4. 特定应用（如冷启动）的需求。

- 迁移学习的基本问题

How to transfer: 如何进行迁移学习? (设计迁移方法)

What to transfer: 给定一个目标领域, 如何找到相对应的源领域, 然后进行迁移? (源领域选择)

When to transfer: 什么时候可以进行迁移, 什么时候不可以? (避免负迁移)

- 迁移学习常用概念

- 基本定义

域(Domain): 数据特征和特征分布组成, 是学习的主体

任务(Task): 由目标函数和学习结果组成, 是学习的结果

- 按特征空间分类

同构迁移学习 (Homogeneous TL): 源域和目标域的特征空间相同, 转存失败重新上传取消 $D_s = D_t$

异构迁移学习 (Heterogeneous TL): 源域和目标域的特征空间不同, 转存失败重新上传取消 $D_s \neq D_t$

- 按迁移情景分类

归纳式迁移学习 (Inductive TL): 源域和目标域的学习任务不同

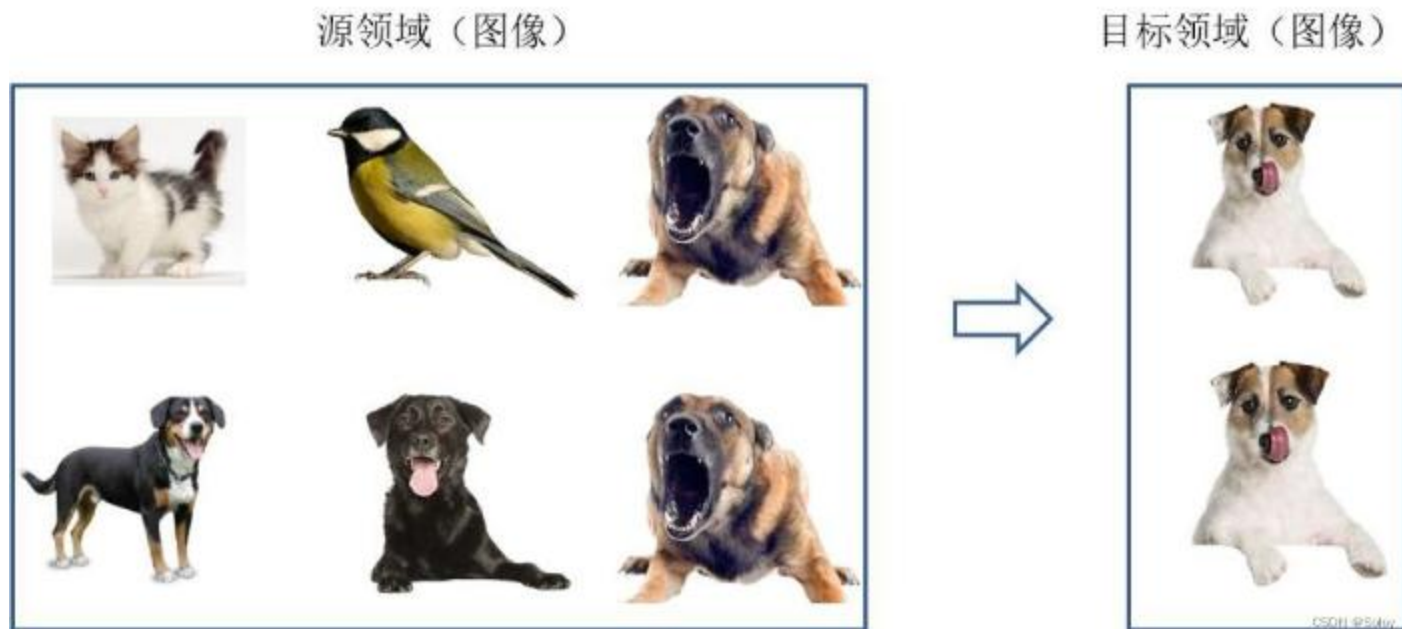
直推式迁移学习 (Transductive TL): 源域和目标域不同, 学习任务相同

无监督迁移学习 (Unsupervised TL): 源域和目标域均没有标签

- 按迁移方法分类

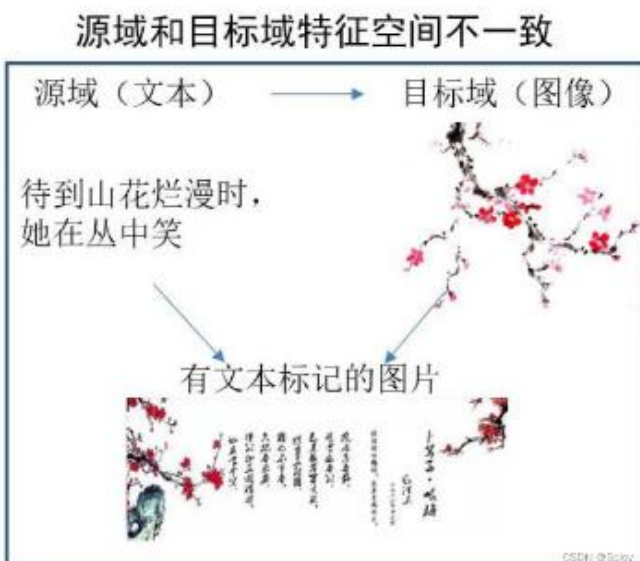
基于样本的迁移 (Instance based TL): 通过权重重用源域和目标域的样例进行迁移。根据一定的权重生成规则, 对数据样本进行重用, 来进行迁移学习。下图形象地表示了基于样本迁移方法的思想

源域中存在不同种类的动物, 如狗、鸟、猫等, 目标域只有狗这一种类别。在迁移时, 为了最大限度地和目标域相似, 我们可以人为地提高源域中属于狗这个类别的样本权重。



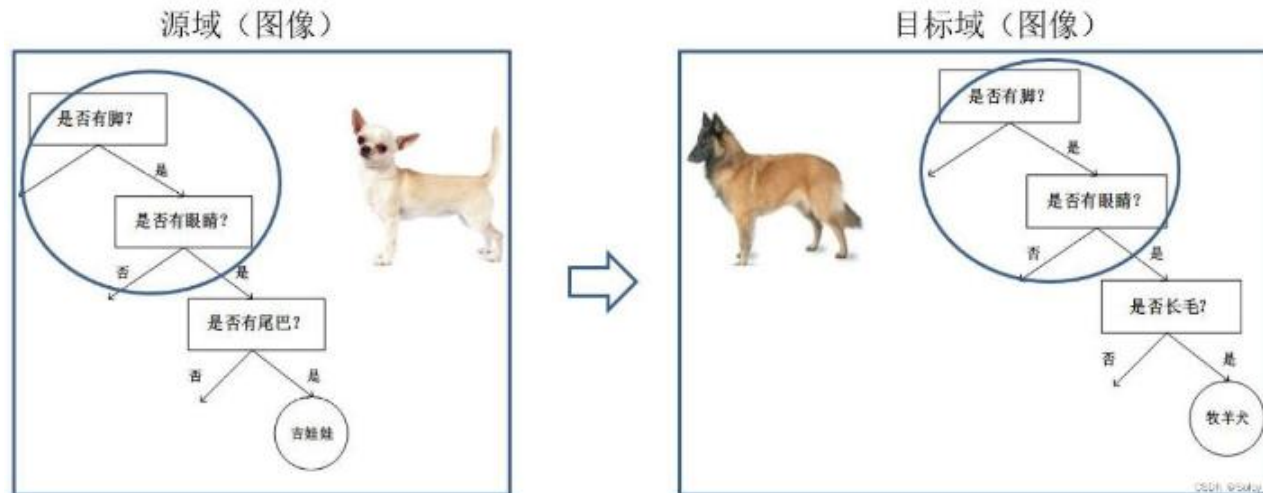
- 按迁移方法分类

基于特征的迁移 (Feature based TL): 将源域和目标域的特征变换到相同空间。是指将通过特征变换的方式互相迁移,来减少源域和目标域之间的差距; 或者将源域和目标域的数据特征变换到统一特征空间中,然后利用传统的机器学习方法进行分类识别。根据特征的同构和异构性,又可以分为同构和异构迁移学习。下图很形象地表示了两种基于特征的迁移学习方法。



- 按迁移方法分类

基于模型的迁移 (Parameter based TL): 利用源域和目标域的参数共享模型。是指从源域和目标域中找到他们之间共享的参数信息,以实现迁移的方法。这种迁移方式要求的假设条件是: 源域中的数据与目标域中的数据可以共享一些模型的参数。下图形象地表示了基于模型的迁移学习方法的基本思想。



- 按迁移方法分类

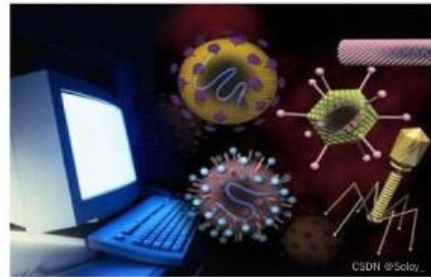
基于关系的迁移 (Relation based TL): 利用源域中的逻辑网络关系进行迁移。与上述三种方法具有截然不同的思路。这种方法比较关注源域和目标域的样本之间的关系。下图形象地表示了不同领域之间相似的关系。



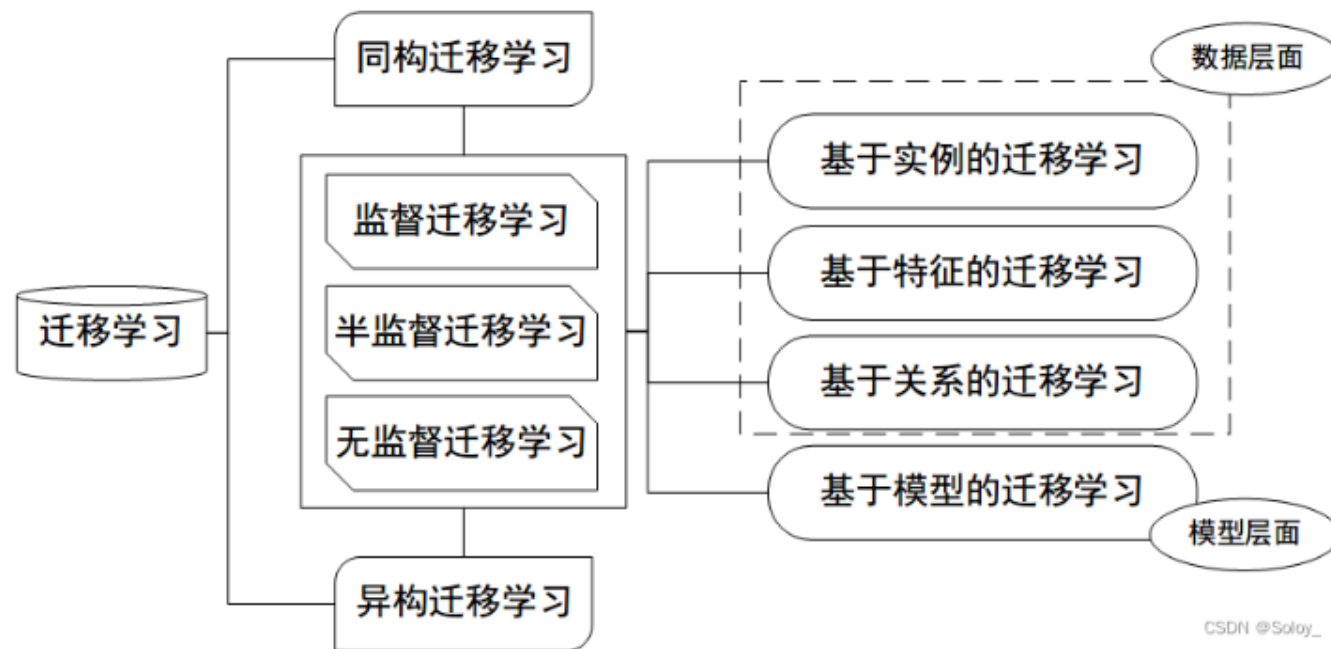
生物病毒



计算机病毒



迁移学习 (Transfer Learning)



	迁移学习	传统机器学习
数据分布	训练和测试数据不需要同分布	训练和测试数据同分布
数据标签	不需要足够的数据标注	足够的数据标注
建模	可以重用之前的模型	每个任务分别建模

- 什么情况下可以使用迁移学习？
- 微调 (finetune) 是什么？
- 什么是深度网络自适应？
- GAN？

实验2: 迁移学习 (Transfer Learning)--迁移学习策略详解

- 实验背景

迁移学习是深度学习中的关键技术, 通过利用预训练模型的知识来提升目标任务的学习效率和性能。本实验将系统对比三种主要迁移学习策略在CIFAR-10数据集上的表现, 帮助同学们理解不同策略的优缺点及适用场景。

- 数据预处理:

调整图像大小至 224×224 (匹配预训练模型输入), 应用标准化处理(使用ImageNet均值和标准差), 训练集应用数据增强: 随机裁剪、随机水平翻转、随机颜色抖动。

airplane

automobile

bird

cat

deer

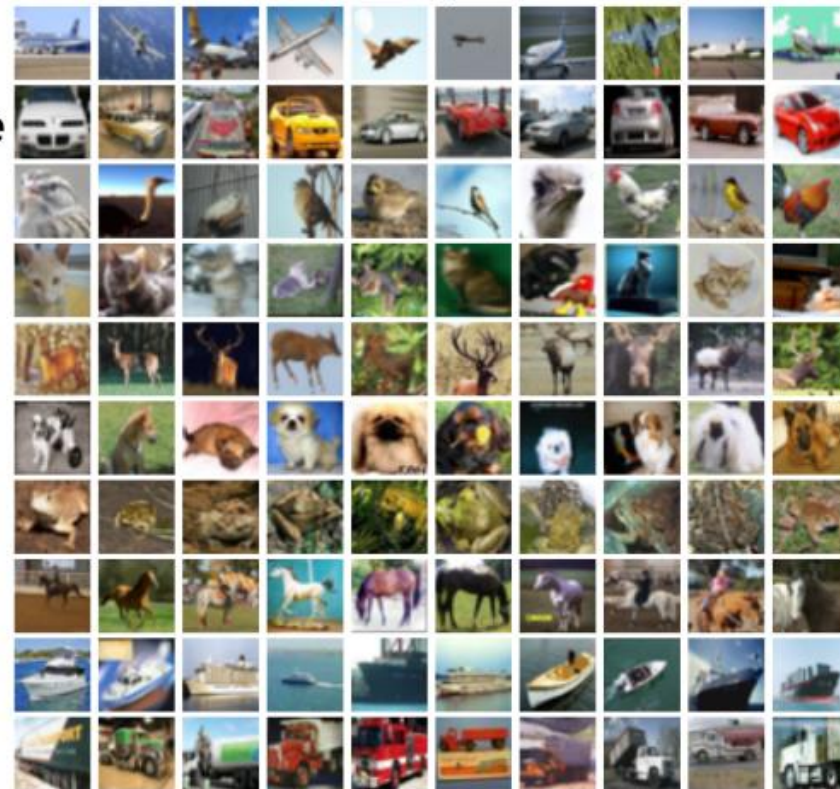
dog

frog

horse

ship

truck



实验2：迁移学习 (Transfer Learning)--迁移学习策略详解

- 特征提取 (Feature Extraction)

原理：将预训练模型视为固定特征提取器，仅训练新添加的分类器层。

实现细节：

1. 加载预训练模型(如ResNet18)
2. 冻结所有卷积层权重 (`requires_grad = False`)
3. 替换并重新初始化最后的全连接层（适配CIFAR-10的10个类别）
4. 仅优化新添加的分类器层参数

训练设置：

1. 学习率: `FEATURE_EXTRACT_LR = 0.001`
2. 优化器: `SGD(动量=0.9)`
3. 学习率调度: `StepLR`(每7个epoch衰减为原来的0.1倍)

实验2: 迁移学习 (Transfer Learning)--迁移学习策略详解

- 微调 (Fine-tuning)

原理: 同时调整预训练模型权重和新分类器, 但使用不同的学习率。

实现细节:

1. 加载预训练模型(如ResNet18)
2. 替换最后的全连接层(适配CIFAR-10的10个类别)
3. 对预训练参数使用较小学习率, 对新分类器使用较大学习率
4. 优化所有模型参数

训练设置:

1. 预训练层学习率: $\text{FINE_TUNE_LR_BACKBONE} = 0.0001$
2. 分类器层学习率: $\text{FINE_TUNE_LR_FC} = 0.001$
3. 优化器: SGD(动量=0.9)
4. 学习率调度: StepLR(每7个epoch衰减为原来的0.1倍)

实验2: 迁移学习 (Transfer Learning)--迁移学习策略详解

- 渐进式解冻 (Gradual Unfreezing)

原理:先解冻并训练顶层, 随着训练进行逐步解冻更深的层, 避免灾难性遗忘。

实现细节:

1. 加载预训练模型(如ResNet18)
2. 将模型分为多个层组(按深度)
3. 初始仅解冻分类器层
4. 每训练GRAD_UNFREEZE_EVERY个epoch后, 解冻下一组层
5. 动态更新优化器以包含新解冻的参数

训练设置:

1. 学习率: `FEATURE_EXTRACT_LR = 0.001`
2. 解冻间隔: `GRAD_UNFREEZE_EVERY = 2`个epoch
3. 优化器: 每次解冻后重新创建, 包含当前所有可训练参数

实验2: 迁移学习 (Transfer Learning)--预训练模型&评估方法

- 支持的预训练模型

- ResNet18:

- 参数量: 约11.7M

- 特点: 残差连接结构, 较深但训练稳定

- MobileNetV2:

- 参数量: 约3.5M

- 特点: 轻量级架构, 倒残差结构, 深度可分离卷积

- EfficientNet-B0:

- 参数量: 约5.3M

- 特点: 复合缩放方法, 均衡网络宽度、深度和分辨率

- 评估指标:

- Top-1准确率

- 每类精确率(Precision)、召回率(Recall)和F1分数

- 混淆矩阵

- 训练和验证损失曲线

实验2：迁移学习 (Transfer Learning)--思考题

- 理论基础思考

--从表示学习的角度看，预训练CNN的不同层学习了什么样的特征？这些特征如何影响迁移学习效果？

--从信息论和模型复杂度角度，解释为什么预训练模型能帮助目标任务更快收敛并达到更好性能。

- 实验结果分析思考

--基于你的实验结果，三种迁移学习策略在哪些方面表现出差异？这些差异与理论预期是否一致？

- 参数敏感性思考

--学习率对三种迁移学习策略的影响是否相同？具体来说，预训练层和新添加层的最佳学习率比例应该是多少？

--在渐进式解冻中，解冻的时机(`GRAD_UNFREEZE EVERY`)如何影响模型性能？理想的解冻策略应考虑哪些因素？

- 实际应用思考

--在资源受限环境中(如移动设备)，如何在模型大小、计算复杂度和迁移学习效果之间做出最佳权衡？

--如果预训练模型的训练数据包含偏见或不平衡表示，这种偏见可能如何通过迁移学习传递到目标任务？如何减轻这种影响？

Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/05/16